FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Leandro Martinez
Section:	AN43
Professor:	Ms. Crisola G. Tan
Date:	October 12, 2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects

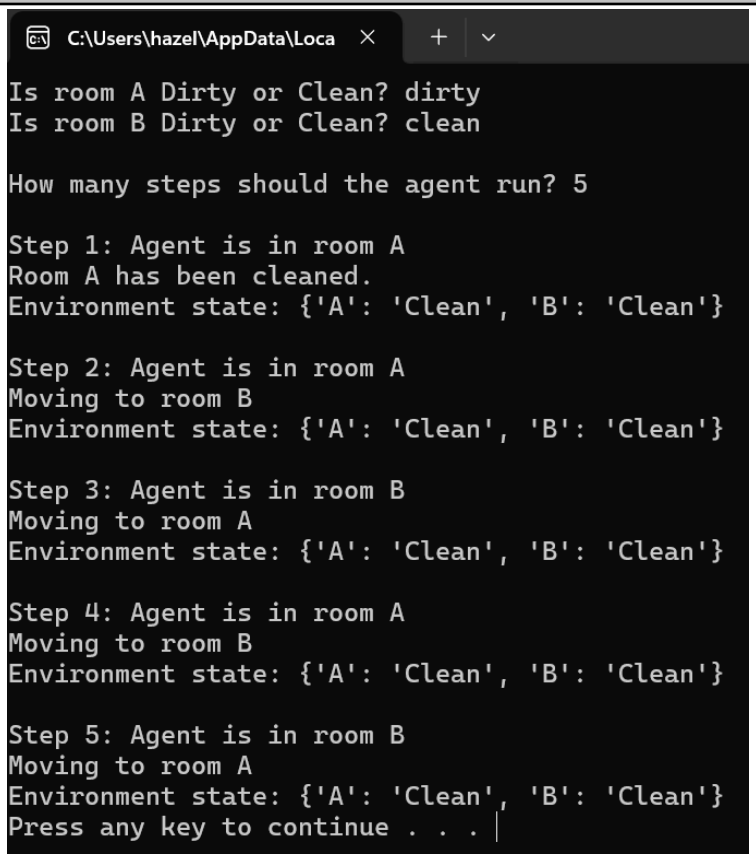
- If-Else statements
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

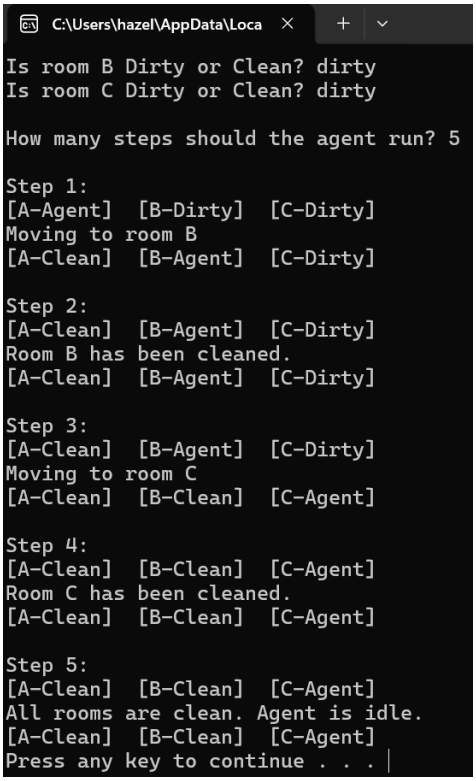
Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

```
# Set up the environment
rooms = {
        "A": input("Enter the initial state of Room A (dirty/clean):
").lower(),
```

```
        "B": input("Enter the initial state of Room B (dirty/clean):
").lower()
}

# Validate room states
for room in rooms:
    while rooms[room] not in ["dirty", "clean"]:
        rooms[room] = input(
            f"Invalid input. Enter the state of Room {room} (dirty/clean):
"
        ).lower()


# Function to check if a room is dirty
def is_dirty(room):
    return rooms[room] == "dirty"


# Function to clean a room
def clean_room(room):
    rooms[room] = "clean"


# Rule-based agent class
class VacuumAgent:
    def __init__(self):
        self.location = "A"

    # Move between Room A and Room B
    def move(self):
        if self.location == "A":
            self.location = "B"
        else:
            self.location = "A"

    # Perceive the room and perform an action
    def perceive_and_act(self):
```

```
        current_room = self.location

        # Rule 1: If the room is dirty, clean it
        if is_dirty(current_room):
            clean_room(current_room)
            action = f"Cleaned Room {current_room}"

        # Rule 2: If the room is clean, move to the other room
        else:
            self.move()
            action = f"Moved to Room {self.location}"

        return action


# Create  agent
agent = VacuumAgent()

# Ask the user how many steps to run
steps = int(input("Enter the number of simulation steps: "))

print("\nInitial Environment:")
print(rooms)
print(f"Initial Agent Location: Room {agent.location}")

# Run  simulation
for step in range(1, steps + 1):
    action = agent.perceive_and_act()

    print(f"\nStep {step}")
    print(f"Agent Location: Room {agent.location}")
    print(f"Action Taken: {action}")
    print(f"Environment: {rooms}")

print("\nSimulation complete.")
```

```
Enter the initial state of Room A (dirty/clean): dirty
Enter the initial state of Room B (dirty/clean): dirty
Enter the number of simulation steps: 5

Initial Environment:
{'A': 'dirty', 'B': 'dirty'}
Initial Agent Location: Room A

Step 1
Agent Location: Room A
Action Taken: Cleaned Room A
Environment: {'A': 'clean', 'B': 'dirty'}

Step 2
Agent Location: Room B
Action Taken: Moved to Room B
Environment: {'A': 'clean', 'B': 'dirty'}

Step 3
Agent Location: Room B
Action Taken: Cleaned Room B
Environment: {'A': 'clean', 'B': 'clean'}
```

```
Step 4
Agent Location: Room A
Action Taken: Moved to Room A
Environment: {'A': 'clean', 'B': 'clean'}

Step 5
Agent Location: Room B
Action Taken: Moved to Room B
Environment: {'A': 'clean', 'B': 'clean'}
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. What rules does the agent follow?

The agent follows two main rules:
- If the current room is dirty, the agent cleans it.
- If the current room is clean, the agent moves to the other room.

2. How does the agent decide when to clean and when to move?

The agent checks the condition of its current room. If the room is dirty, it cleans it. If the room is already clean, it moves to the other room.

3. What happens if both rooms are already clean at the start?

Since there is nothing to clean, the agent will simply move between Room A and Room B during each step.

4. How would you modify the agent to handle more than two rooms?

I would store the rooms in a list or dictionary and use an index to track the agent's location. When the current room is clean, the agent can move to the next room using:
current_index = (current_index + 1) % len(rooms)